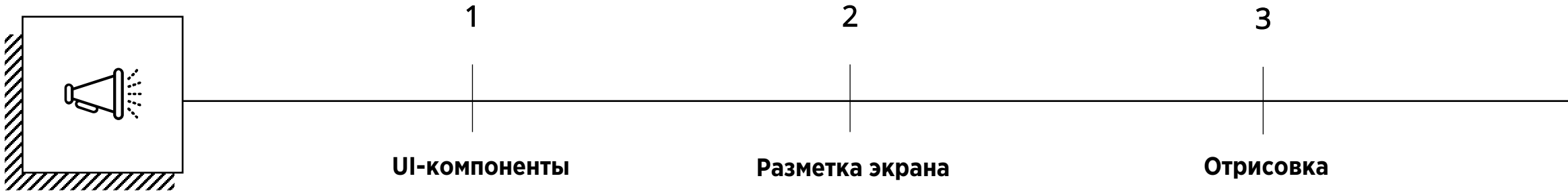
Разметка и UI-компоненты

Содержание лекции



UI-компоненты

Для каждой платформы существует набор готовых компонент пользовательского интерфейса. Система Android не исключение и обладает большим набором таких компонент.

В предыдущих курсах мы рассматривали какие есть UI-компоненты и для чего они нужны. Сейчас же быстро пройдемся по основным элементам и посмотрим, что они из себя представляют.

UI-компоненты

Все компоненты и макетов наследуются от двух базовых классов.

- View – базовый класс для всех UI-компонент
- ViewGroup – базовый класс для макетов, дочерний от View

UI-компоненты

По факту, View является, с одной стороны, базовым классом для всех представлений (элементов UI), а с другой, элементом-заглушкой для построения пользовательского интерфейса.

Данный компонент представляет собой прямоугольник и отвечает за отрисовку элемента и управление событиями.

У класса View есть еще один наследник - ViewGroup. Класс ViewGroup позволяет хранить в себе другие экземпляры класса View. Он является базовым для большинства контейнеров представлений и макетов разметки (layout).

UI-компоненты

UI-компоненты это обычные классы, у которых есть атрибуты. Но с ними можно работать двумя способами:

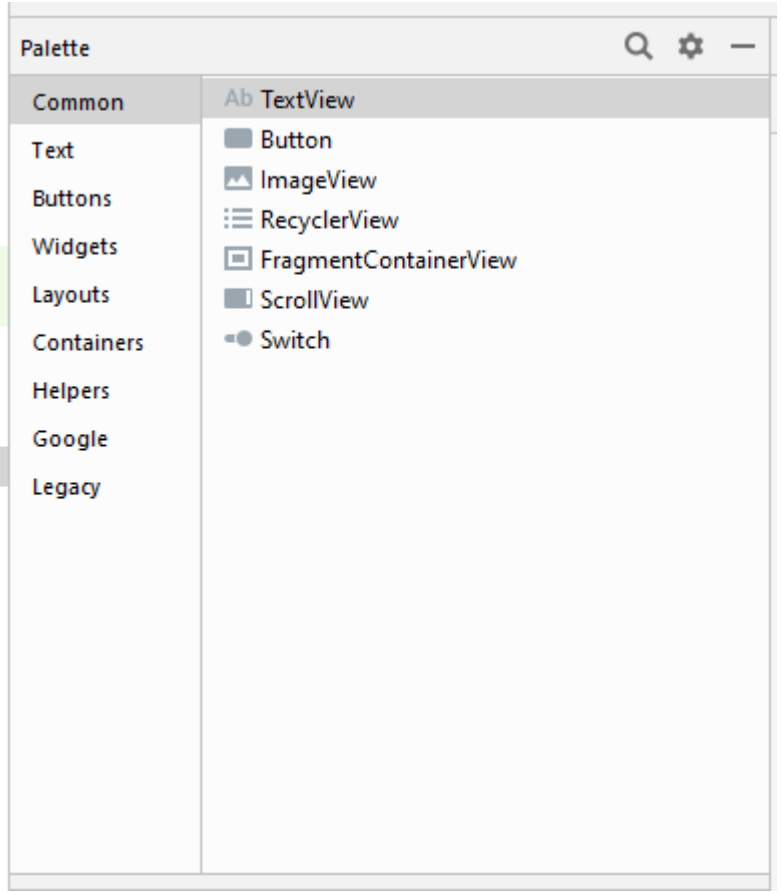
- С помощью XML-разметки
- Напрямую из кода программы

Зачастую это комбинированный подход, когда мы размечаем основные элементы с помощью XML-разметки и работаем с компонентами уже непосредственно из кода.

UI-компоненты

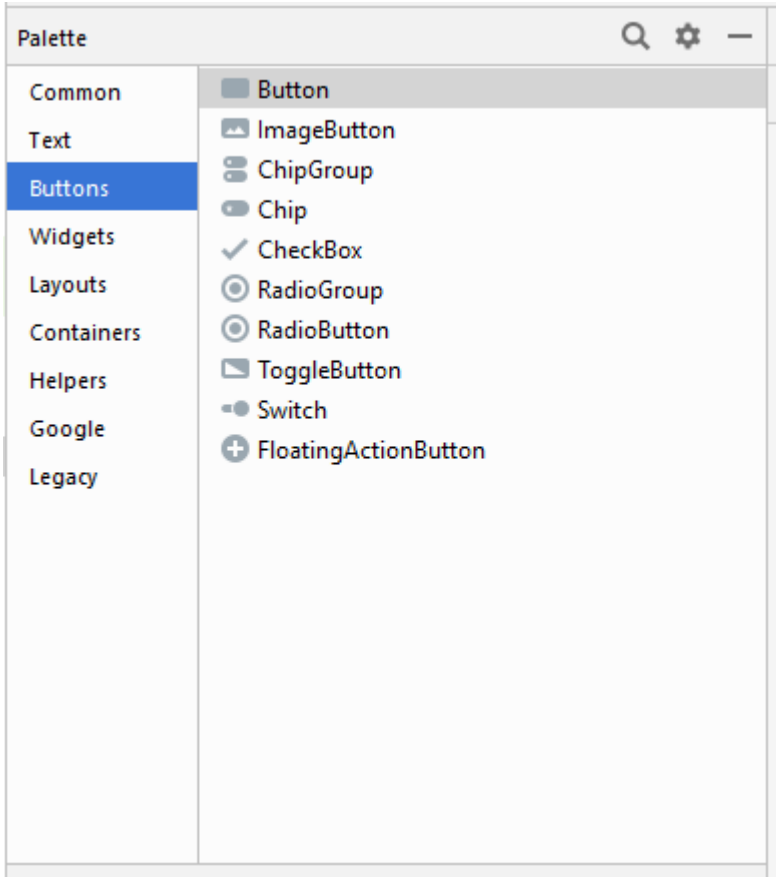
В Android Studio компоненты разбиваются на категории:

- Text
- Buttons
- Widgets
- Layouts
- Containers
- Helpers
- Google
- Legacy



UI-компоненты

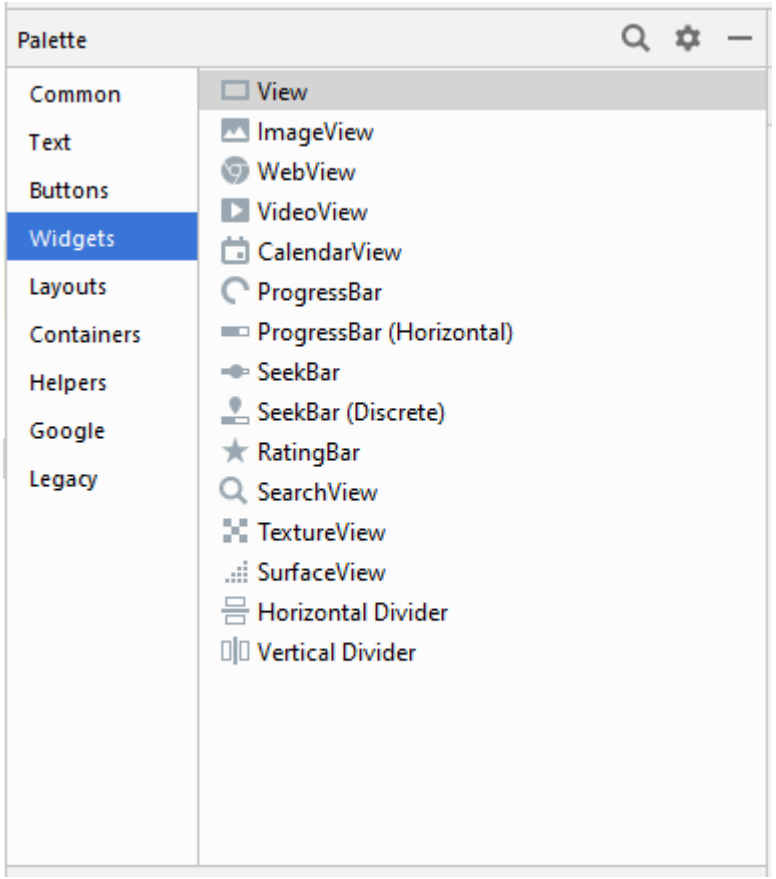
Категория Buttons содержит все компоненты кнопок, которые можно использовать в Android.



UI-компоненты

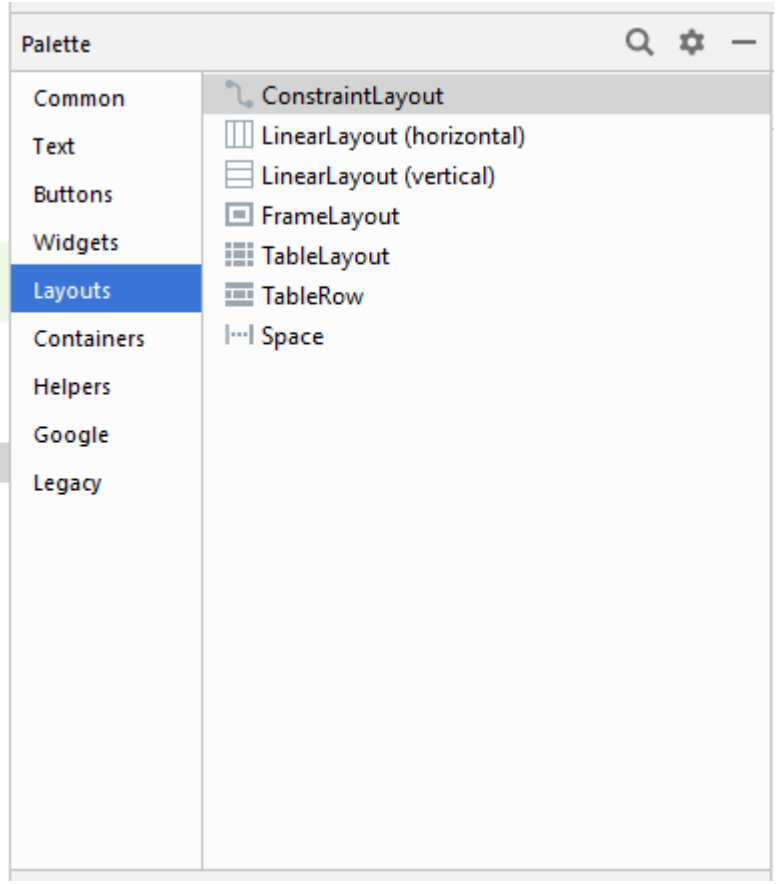
Категория Widgets содержит все компоненты для отображения различно контента и различные инструменты настройки интерфейса.

Данная категория содержит и компонент View.



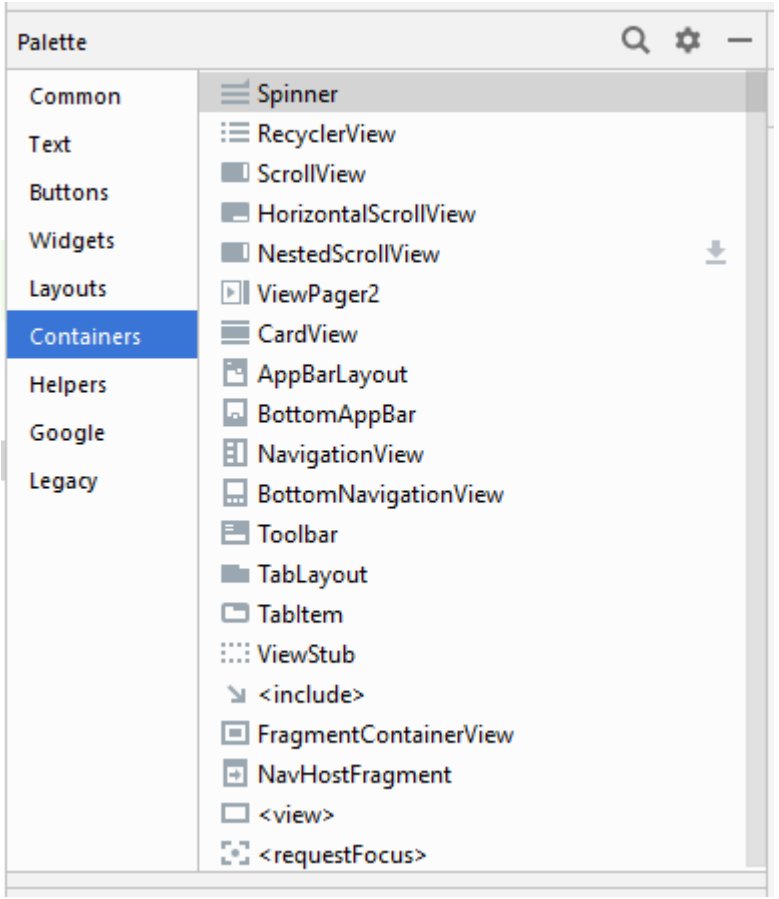
UI-компоненты

Категория Layout содержит макеты для разметки сложных интерфейсов.



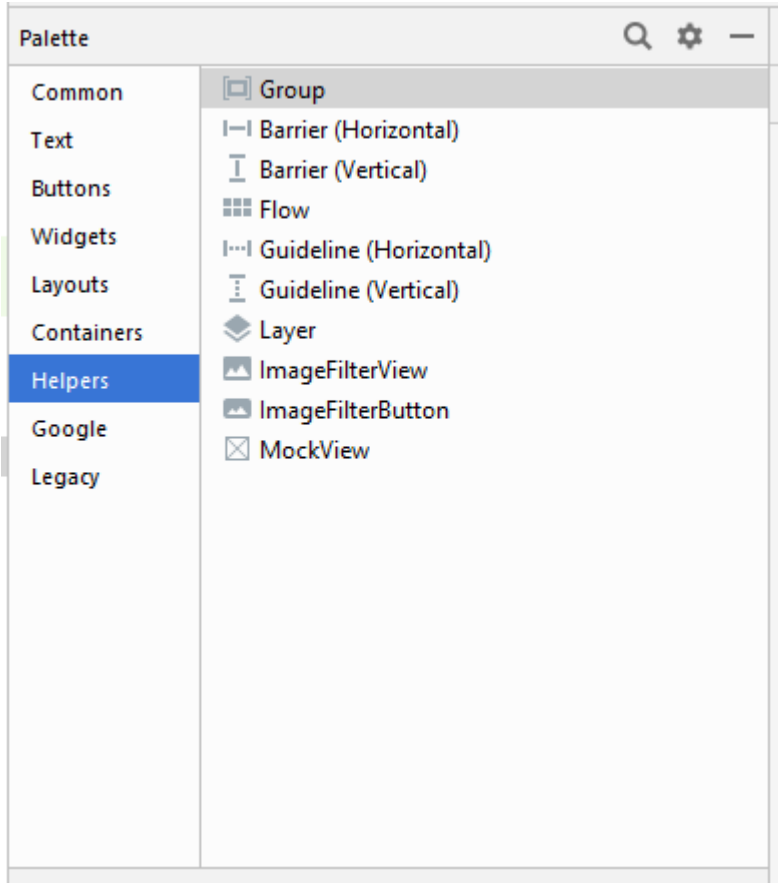
UI-компоненты

Категория Containers содержит все контейнеры для и элементы навигации приложения.



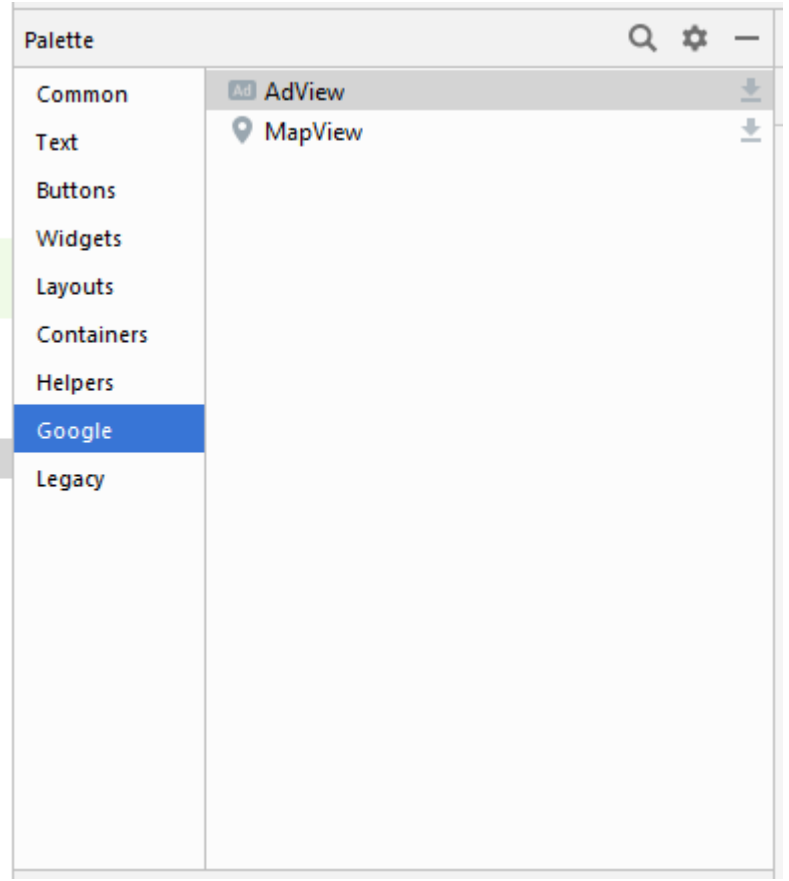
UI-компоненты

Категория Helpers содержит различные вспомогательные компоненты более точной настройки интерфейса если мы работаем с ConstraintLayout.



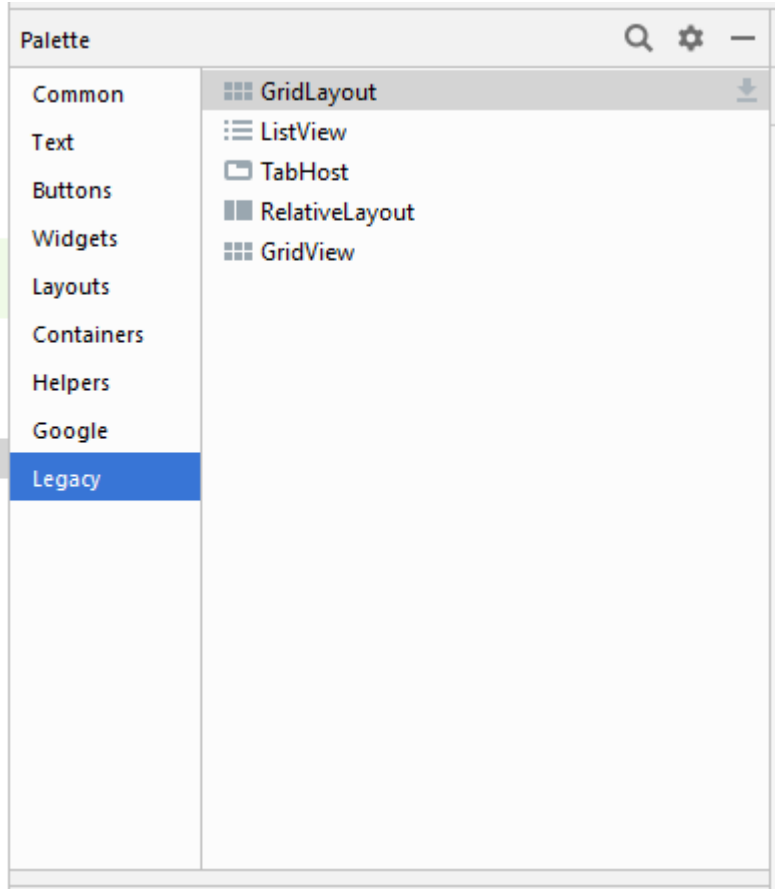
UI-компоненты

Категория Google содержит компоненты для рекламных баннеров и карты.



UI-компоненты

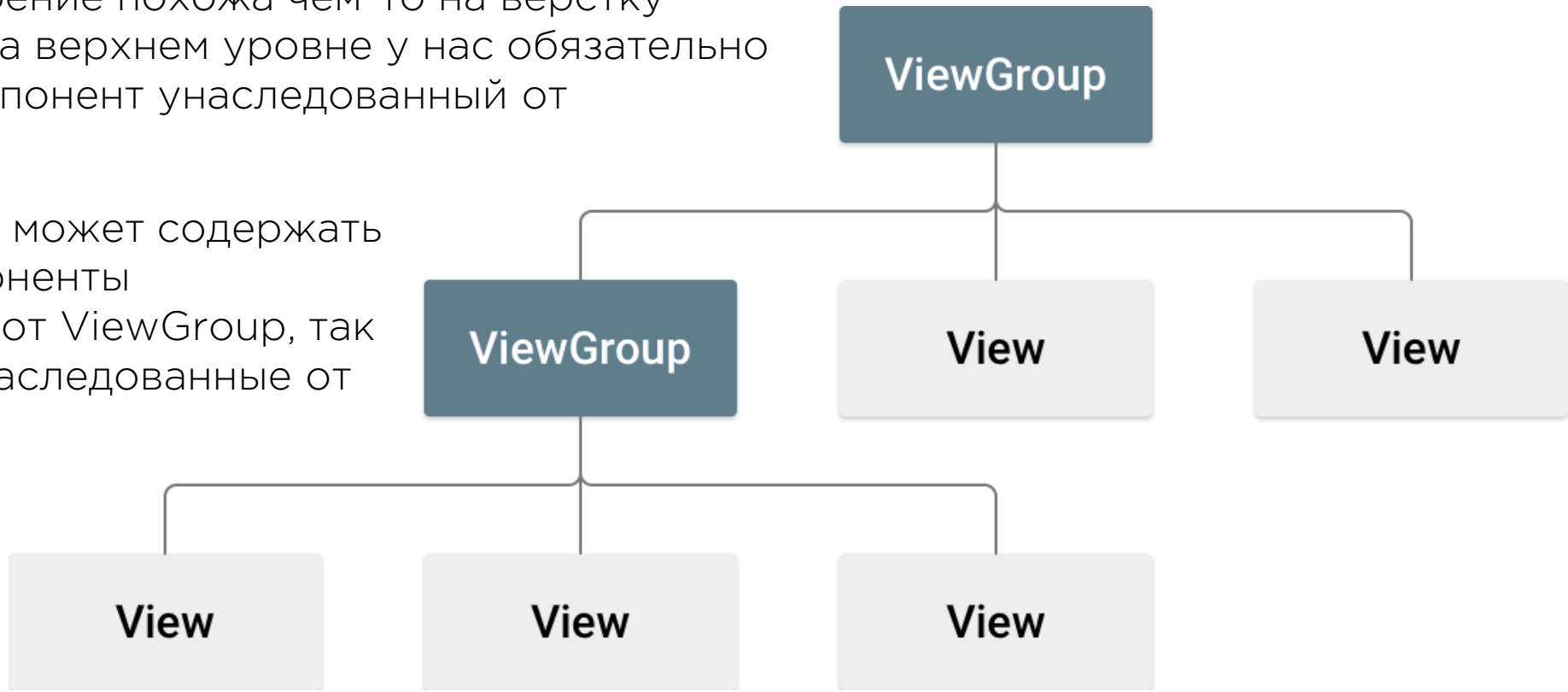
Категория Legacy содержит все компоненты, которые уже являются устаревшими, но все еще могут использоваться.



Разметка экрана

Иерархия построение похожа чем-то на верстку HTML-страниц. На верхнем уровне у нас обязательно должен быть компонент унаследованный от ViewGroup.

Такой компонент может содержать как другие компоненты унаследованные от ViewGroup, так и компоненты унаследованные от View.



Разметка экрана

Объявление вашего пользовательского интерфейса в XML позволяет отделить представление вашего приложения от кода, который контролирует его поведение. Использование файлов XML также упрощает предоставление различных макетов для разных размеров и ориентации экрана.

Каждый файл макета должен содержать ровно один корневой элемент, который должен быть объектом View или ViewGroup. После того, как вы определили корневой элемент, вы можете добавить дополнительные объекты макета или виджеты в качестве дочерних элементов для постепенного построения иерархии представлений, определяющей ваш макет.

Разметка экрана

Например, вот XML-макет, который использует вертикальный LinearLayout для хранения TextView и Button:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
              android:layout_width="match_parent"
              android:layout_height="match_parent"
              android:orientation="vertical" >
    <TextView android:id="@+id/text"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Hello, I am a Button" />
</LinearLayout>
```

Разметка экрана

При компиляции приложения каждый файл макета XML компилируется в ресурс представления. Ресурс макета следует загрузить из кода приложения в реализацию обратного вызова Activity.onCreate ().

Для загрузки необходимо вызвать метод setContentView (), передав ему ссылку на ваш ресурс макета в виде: R.layout.layout_file_name.

Например, если ваш XML-макет сохранен как main_layout.xml, вы должны загрузить его для своей Activity следующим образом:

```
fun onCreate(savedInstanceState: Bundle) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.main_layout)
}
```

Разметка экрана

Каждый объект View и ViewGroup поддерживает свой набор атрибутов XML. Некоторые атрибуты специфичны для объекта View (например, TextView поддерживает атрибут textSize), но эти атрибуты также наследуются любыми объектами View, которые могут расширять этот класс. Некоторые из них являются общими для всех объектов View, поскольку они унаследованы от корневого класса View (например, атрибут id).

И другие атрибуты считаются «параметрами макета», которые являются атрибутами, которые описывают определенные ориентации макета объекта View, как определено родительским объектом ViewGroup этого объекта.

Разметка экрана

Любой объект View может иметь связанный с ним целочисленный идентификатор, чтобы однозначно идентифицировать представление в дереве. При компиляции приложения на этот идентификатор ссылаются как на целое число, но идентификатор обычно назначается в XML-файле макета в виде строки в атрибуте id. Это атрибут XML, общий для всех объектов View (определенный классом View).

Синтаксис идентификатора внутри тега XML:

```
android:id="@+id/my_button"
```

Разметка экрана

Символ собаки (@) в начале строки указывает, что синтаксический анализатор XML должен проанализировать и развернуть оставшуюся часть строки идентификатора и идентифицировать ее как ресурс идентификатора. Знак плюс (+) означает, что это новое имя ресурса, которое необходимо создать и добавить к нашим ресурсам (в файле R.java).

Платформа Android предлагает ряд других ресурсов ID. При ссылке на идентификатор ресурса Android знак плюса не требуется, но необходимо добавить пространство имен пакета Android, например:

```
android:id="@android:id/empty"
```

Разметка экрана

Имея пространство имен пакетов android, мы теперь ссылаемся на идентификатор из класса ресурсов android.R, а не из класса локальных ресурсов.

Чтобы создавать представления и ссылаться на них из приложения, в общем случае необходимо сделать следующее:

1. Определите представление / виджет в файле макета и назначьте ему уникальный идентификатор:

```
<Button android:id="@+id/my_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/my_button_text"/>
```

Разметка экрана

2. Затем создайте экземпляр объекта View и захватите его из макета (обычно в методе onCreate ()):

```
val myButton: Button = findViewById(R.id.my_button)
```

Разметка экрана

Определение идентификаторов для объектов View важно при создании RelativeLayout. В относительном макете одноуровневые View могут определять свой макет относительно другого одноуровневого View, на которое ссылается уникальный идентификатор.

Идентификатор не обязательно должен быть уникальным во всем дереве, но он должен быть уникальным в пределах той части дерева, которую вы рассматриваете (которая часто может быть всем деревом, поэтому по возможности лучше использовать полностью уникальный ID).

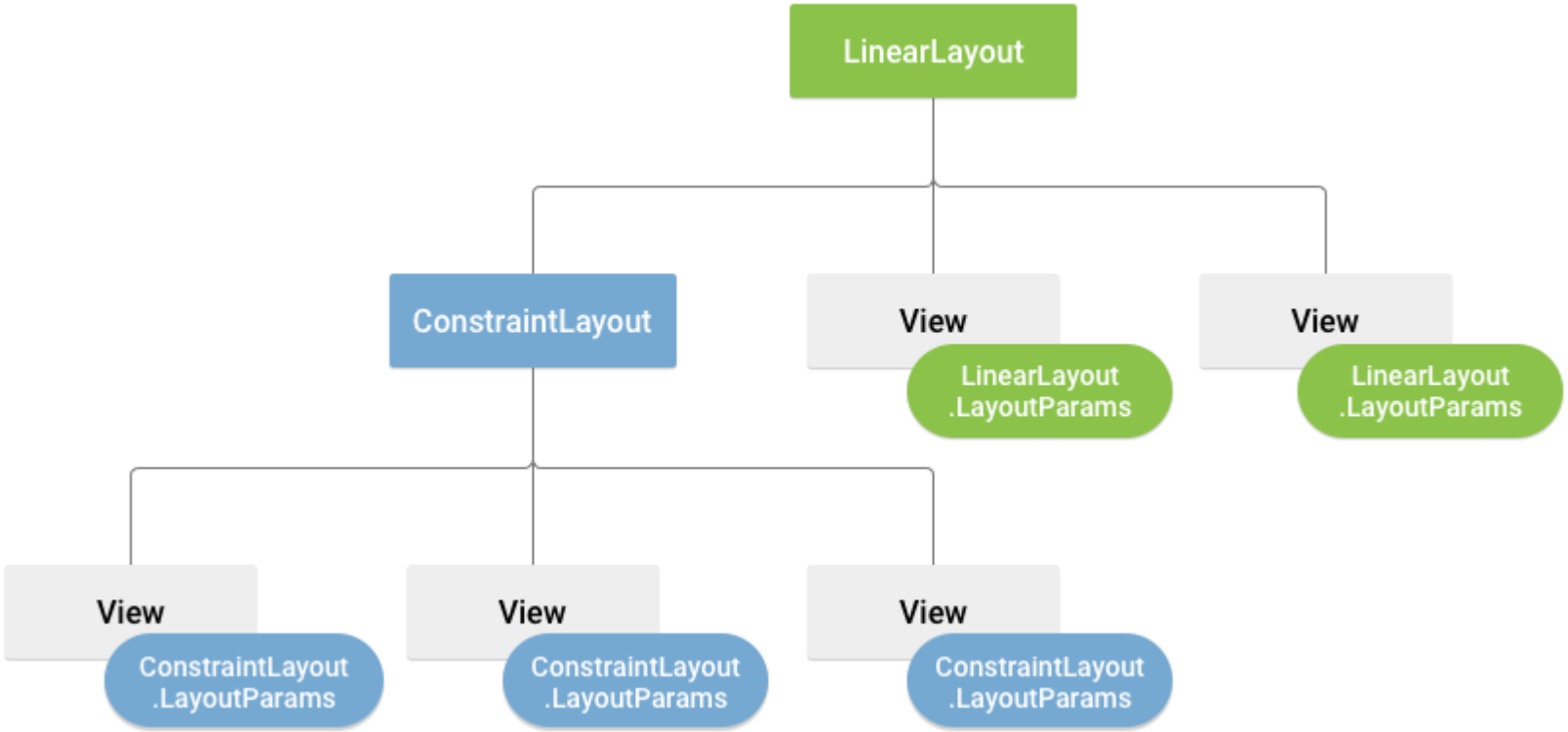
Разметка экрана

Атрибуты макета XML с именем layout_something определяют параметры макета для View, которые подходят для ViewGroup, в которой он находится.

Каждый класс ViewGroup реализует вложенный класс, расширяющий ViewGroup.LayoutParams. Этот вложенный класс содержит типы свойств, которые определяют размер и положение для каждого дочернего View в зависимости от группы View.

Отрисовка

Родительская ViewGroup определяет параметры макета для каждого дочернего View (включая дочернюю ViewGroup).



Отрисовка

Обратите внимание, что каждый подкласс LayoutParams имеет собственный синтаксис для установки значений. Каждый дочерний элемент должен определять LayoutParams, соответствующий его родительскому элементу, хотя он также может определять разные LayoutParams для своих собственных дочерних элементов.

Все ViewGroup включают ширину и высоту (layout_width и layout_height), и каждое View должно их определить. Многие LayoutParams также включают необязательные поля и границы.

Отрисовка

Вы можете указать ширину и высоту с точными размерами. Чаще всего вы будете использовать одну из этих констант для установки ширины или высоты:

- wrap_content – сообщает вашему View, что его размер должен соответствовать размерам, требуемым для его содержимого.

- match_parent – сообщает, что ваше View должно стать настолько большим, насколько позволяет его родительская ViewGroup.

Как правило, не рекомендуется указывать ширину и высоту макета с использованием абсолютных единиц, таких как пиксели. Вместо этого используют относительные измерения, такие как независимые от плотности пиксельные единицы (dp), wrap_content или match_parent, является лучшим подходом, поскольку это помогает гарантировать, что ваше приложение будет правильно отображаться на экранах различных размеров устройства.

Отрисовка

Геометрия View - это прямоугольник. У View есть местоположение, выраженное парой левой и верхней координат, и два измерения, выраженные шириной и высотой. Единицей измерения местоположения и размеров является пиксель. Можно получить местоположение представления, вызвав методы getLeft () и getTop (). Первый возвращает левую или X координату прямоугольника, представляющего View. Последний возвращает координату вершины или Y прямоугольника, представляющего View. Оба эти метода возвращают расположение представления относительно его родителя.

Отрисовка

Например, когда getLeft () возвращает 20, это означает, что представление расположено на 20 пикселей справа от левого края своего прямого родителя. Кроме того, предлагается несколько удобных методов, позволяющих избежать ненужных вычислений, а именно getRight () и getBottom (). Эти методы возвращают координаты правого и нижнего краев прямоугольника, представляющего View. Например, вызов getRight () аналогичен следующему вычислению: getLeft () + getWidth ().

Отрисовка

Размер представления выражается шириной и высотой. Представление фактически имеет две пары значений ширины и высоты.

Первая пара известна как измеренная ширина и измеренная высота. Эти размеры определяют, насколько большим должно быть представление в пределах своего родителя. Измеренные размеры можно получить, вызвав getMeasuredWidth () и getMeasuredHeight ().

Отрисовка

Вторая пара просто известна как ширина и высота или иногда ширина отрисовки и высота отрисовки. Эти размеры определяют фактический размер вида на экране во время отрисовки и после размещения в макете. Эти значения могут, но не обязательно, отличаться от измеренных ширины и высоты. Ширину и высоту можно получить, вызвав getWidth () и getHeight ().Чтобы измерить свои размеры, View учитывает его заполнение. Заполнение выражается в пикселях для левой, верхней, правой и нижней частей View.

Отрисовка

Padding может использоваться для смещения содержимого View на определенное количество пикселей. Например, левое заполнение 2 сдвинет содержимое представления на 2 пикселя вправо от левого края.

Заполнение можно установить с помощью метода setPadding (int, int, int, int) и запросить путем вызова getPaddingLeft (), getPaddingTop (), getPaddingRight () и getPaddingBottom ().Несмотря на то, что View может определять отступы, оно не поддерживает поля. Однако группы представлений предоставляют такую поддержку.

Отрисовка

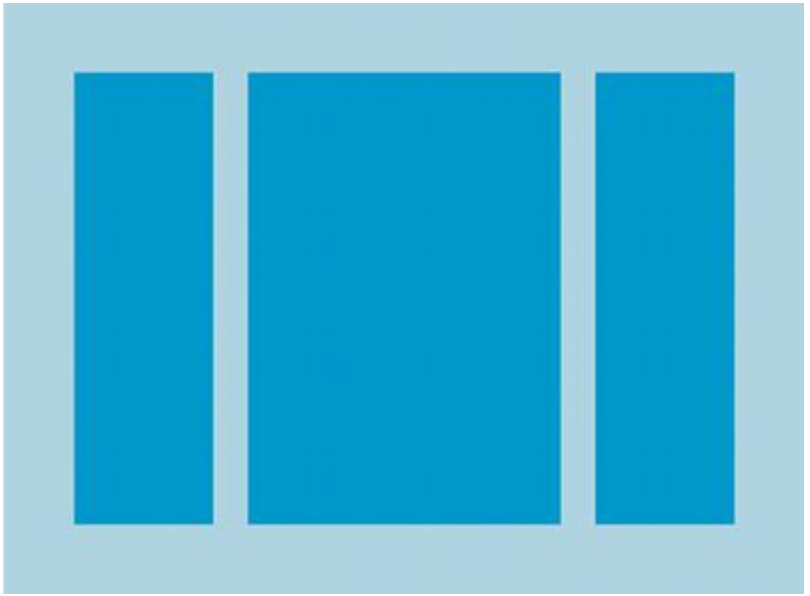
Каждый дочерний класс класса ViewGroup предоставляет уникальный способ отображения вложенных в него View.



Отрисовка

LinearLayout.

Макет, который объединяет дочерние элементы в одну горизонтальную или вертикальную строку. Он создает полосу прокрутки, если длина окна превышает длину экрана.



Отрисовка

RelativeLayout.

Позволяет указать расположение дочерних объектов относительно друг друга (дочерний элемент A слева от дочернего элемента B) или по отношению к родительскому объекту (выровнен по верхнему краю родительского объекта).



Отрисовка

WebView.

Отображает веб-страницы. Не является браузером!

```
<html>

   <!-- web page -->

</html>
```

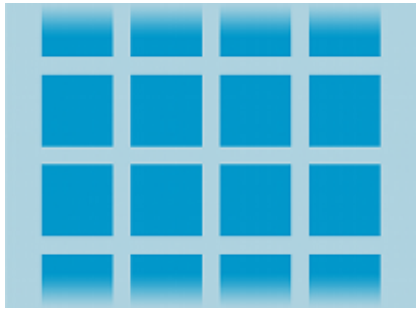
Отрисовка

Если содержимое вашего макета является динамическим или не определено заранее, вы можете использовать макет, который является дочерним классом AdapterView, чтобы заполнить макет представлениями во время выполнения. Дочерний класс класса AdapterView использует адаптер для привязки данных к своему макету. Адаптер действует как посредник между источником данных и макетом AdapterView - адаптер извлекает данные (из источника, такого как массив или запрос к базе данных) и преобразует каждую запись в представление, которое можно добавить в макет AdapterView.

Отрисовка

Общие макеты, поддерживаемые адаптером, включают:

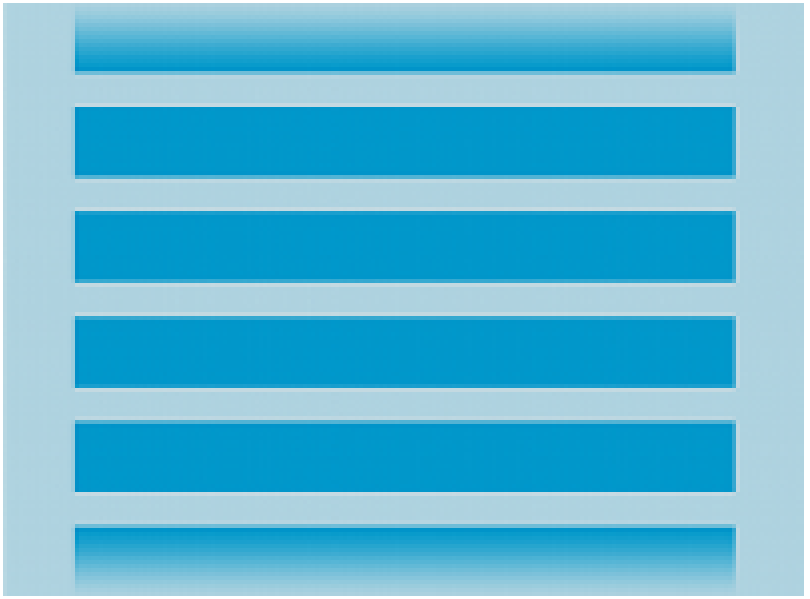




Отрисовка

ListView.

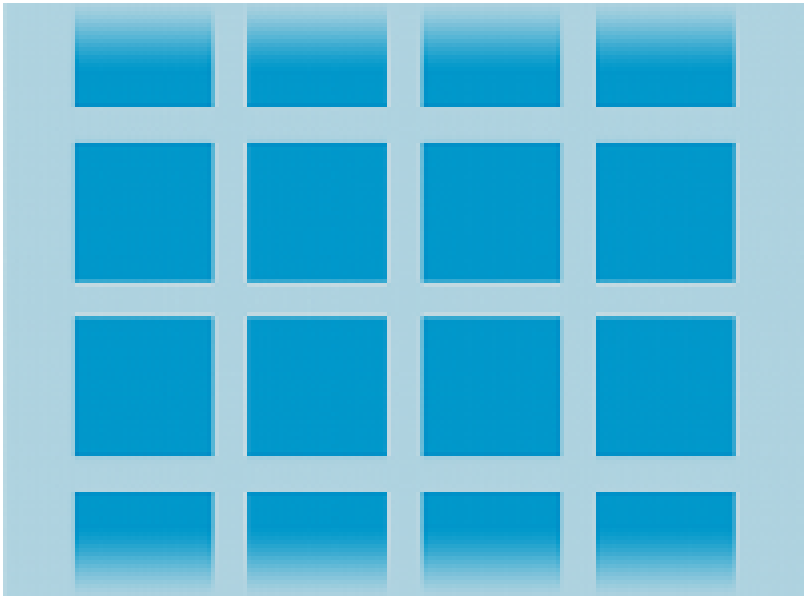
Отображает прокручиваемый список из одного столбца.



Отрисовка

GridView.

Отображает прокручиваемую сетку столбцов и строк.



Отрисовка

Вы можете заполнить AdapterView, например ListView или GridView, привязав экземпляр AdapterView к адаптеру, который извлекает данные из внешнего источника и создает View, представляющее каждую запись данных.

Android предоставляет несколько подклассов Adapter, которые полезны для получения различных типов данных и построения View для AdapterView. Два наиболее распространенных адаптера:

- ArrayAdapter – используйте этот адаптер, если источником данных является массив. По умолчанию ArrayAdapter создает представление для каждого элемента массива, вызывая toString () для каждого элемента и помещая содержимое в TextView.

Отрисовка

Например, если у вас есть массив строк, который вы хотите отобразить в ListView, инициализируйте новый ArrayAdapter с помощью конструктора, чтобы указать макет для каждой строки и массива строк:

```
val adapter = ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, myStringArray)
```

Аргументы для этого конструктора:

- Контекст (Context) вашего приложения
- Макет, содержащий TextView для каждой строки в массиве
- Строковый массив

Отрисовка

Затем просто вызовите setAdapter () в своем ListView:

```
val listView: ListView = findViewById(R.id.listview)
listView.adapter = adapter
```

Отрисовка

Чтобы настроить внешний вид каждого элемента, вы можете переопределить метод toString () для объектов в вашем массиве. Или, чтобы создать View для каждого элемента, отличного от TextView (например, если вам нужен ImageView для каждого элемента массива), расширьте класс ArrayAdapter и переопределите getView (), чтобы вернуть тип View, который вы хотите для каждого элемента.

Отрисовка

- SimpleCursorAdapter – используйте этот адаптер, когда ваши данные поступают из курсора. При использовании SimpleCursorAdapter необходимо указать макет, который будет использоваться для каждой строки в Cursor, и какие столбцы в Cursor должны быть вставлены в какие представления макета.

Отрисовка

Например, если вы хотите создать список имен и телефонных номеров людей, вы можете выполнить запрос, который возвращает Cursor, содержащий строку для каждого человека и столбцы для имен и номеров.

Затем вы создаете массив строк, определяющий, какие столбцы из курсора вы хотите в макете для каждого результата, и целочисленный массив, определяющий соответствующие View, в которые должен быть помещен каждый столбец:

```
val fromColumns = arrayOf(ContactsContract.Data.DISPLAY_NAME,
                          ContactsContract.CommonDataKinds.Phone.NUMBER)
val toViews = intArrayOf(R.id.display_name, R.id.phone_number)
```

Отрисовка

Когда вы создаете экземпляр SimpleCursorAdapter, передайте макет для использования для каждого результата, курсор, содержащий результаты, и эти два массива:

```
val adapter = SimpleCursorAdapter(this,
        R.layout.person_name_and_number, cursor, fromColumns, toViews, 0)
val listView = getListView()
listView.adapter = adapter
```

Затем SimpleCursorAdapter создает View для каждой строки в Cursor, используя предоставленный макет, вставляя каждый элемент fromColumns в соответствующее представление toViews.

Отрисовка

Если в течение жизненного цикла вашего приложения вы меняете базовые данные, которые читает ваш адаптер, вы должны вызвать notifyDataSetChanged (). Это уведомит прикрепленное View о том, что данные были изменены, и оно должно обновиться.

Отрисовка

Вы можете реагировать на события клика по каждому элементу в AdapterView, реализуя интерфейс AdapterView.OnItemClickListener.

Например:

```
listView.onItemClickListener = AdapterView.OnItemClickListener { parent, view, position, id ->
    // Do something in response to the click
}
```

Отрисовка

Context.

Интерфейс к глобальной информации о среде приложения. Это абстрактный класс, реализация которого обеспечивается системой Android.

Он позволяет получить доступ к ресурсам и классам, специфичным для приложения, а также выполнять вызовы для операций на уровне приложения, таких как:

- запуск activity
- broadcasting
- receiving intents
- etc.